The Journal of Open Source Software

earthaccess: A Python Library for Simplified Discovery and Access of NASA Earth Science Data

Andrew P. Barrett [1], **Chris Battisto** [2], **James Bourbeau** [3], **Ian Carroll** [4,14], **Chuck Daniels**[5], **Matt Fisher** [6], **Daniel E. Kaufman** [7,8], **Joseph H. Kennedy** [9], **Luis Lopez** [1], **Julia S. Lowndes** [10], **Jessica Scheick** [11,12], **Amy Steiker** [1], **and Sherwin Varghese** [13]

1 National Snow and Ice Data Center (NSIDC), Cooperative Institute for Research in Environmental Sciences (CIRES), University of Colorado Boulder, Boulder, CO, USA **2** NASA Goddard Space Flight Center, Goddard Earth Sciences Data and Information Services Center (GES DISC), Greenbelt, MD, USA **3** Coiled Computing, Inc. **4** Ocean Ecology Lab, NASA Goddard Space Flight Center, Greenbelt, MD, USA **5** Development Seed **6** Schmidt Center for Data Science and Environment (DSE), University of California Berkeley **7** NASA Langley Research Center, Atmospheric Science Data Center (ASDC), Hampton, VA, USA **8** Booz Allen Hamilton, Inc., McLean, VA, USA **9** Alaska Satellite Facility, Geophysical Institute, University of Alaska Fairbanks, Fairbanks, AK, USA **10** Openscapes and National Center for Ecological Analysis and Synthesis (NCEAS), University of California Santa Barbara, Santa Barbara, CA, USA **11** University of New Hampshire, Durham, NH, USA **12** eScience Institute, University of Washington, Seattle, WA, USA **13** Independent Contributor **14** GESTAR II, University of Maryland Baltimore County, Baltimore, MD, USA

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor:

Submitted: 30 October 2026
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

DRAFT

Summary

earthaccess is an open-source Python library that simplifies discovery, authentication, and access of NASA Earth science data. NASA's Earth Science Data and Information System (ESDIS) Project distributes over 100 petabytes of data across 11 Distributed Active Archive Centers (DAACs) (National Aeronautics and Space Administration, 2026d, 2026c), encompassing satellite imagery, climate records, atmospheric measurements, and other geospatial datasets critical to Earth science research. Accessing these data programmatically has historically required researchers to navigate multiple authentication systems, understand provider-specific APIs and protocols, and write substantial boilerplate code – challenges that particularly deter researchers without deep software engineering experience.

earthaccess provides a unified, high-level Python interface that simplifies this workflow to just a few lines of code. The library handles authentication with NASA's Earthdata Login (EDL) service (National Aeronautics and Space Administration, 2026b), queries to NASA's Common Metadata Repository (CMR) (National Aeronautics and Space Administration, 2026a) for data discovery, and data retrieval through a single interface covering both HTTPS download and direct Amazon Web Services (AWS) S3 access. earthaccess also supports streaming data directly into analysis-ready formats using `fsspec` (fsspec Contributors, 2026) and constructing virtual Zarr stores from archival formats (e.g., HDF5 and NetCDF4) using DMR++ metadata (OPeNDAP, 2026) (powered by `VirtualiZarr` (Nicholas et al., 2026) and `kerchunk` (kerchunk Contributors, 2026)), enabling drastic improvements in access performance.

Statement of need

NASA's Earth science data archive is one of the largest and most diverse collections of Earth observation data in the world, used by over 14 million researchers, educators, and decision-makers globally (National Aeronautics and Space Administration, 2026c). However,

the complexity of the underlying data infrastructure presents a significant barrier to scientific productivity. A typical data access workflow requires a researcher to: (1) search the CMR API to discover relevant datasets and products; (2) parse the returned metadata to identify specific granules and extract the appropriate access URLs; (3) authenticate with NASA Earthdata Login while maintaining authenticated sessions that handle cross-domain redirects; (4) obtain and periodically renew temporary AWS S3 credentials when computing in the same AWS region as the data; and (5) retrieve the data by downloading it over HTTPS, streaming it directly from S3, or passing the information into a third-party package for customization services. Researchers must know enough about the technical implementation of multiple unique APIs to perform these steps accurately, each of which introduces opportunities for error. DAAC-specific configurations and NASA's migration of data to the Earthdata Cloud compound the challenge.

NASA's Earthdata Cloud introduced a novel source of complexity for most researchers, who must now choose between two access paradigms: downloading data over the public internet, and direct S3 access from within the same AWS region (`us-west-2`), which offers higher throughput and avoids data egress costs. Both paradigms use HTTP; they differ in how requests are routed and billed, and both may even occur within a single analysis workflow. During workshops organized by NASA Openscapes (Lowndes et al., 2019; Lowndes & Robinson, 2026), the need for simpler tools became evident. Meanwhile, developers supporting these communities were writing the same access code across many applications and repairing each copy whenever CMR changed.

`earthaccess` is a community project that was created to address this gap: it provides a uniform interface for access to NASA Earthdata regardless of data storage location and handles authentication, credentials, and token management behind the scenes, enabling researchers to focus more on scientific interpretation and discovery.

The target audience spans Earth system science and its applications, including atmospheric scientists, oceanographers, ecologists, land surface modelers, remote sensing researchers, weather forecasters, disaster response practitioners, civic decision-makers, educators, and application developers. The library is designed to be approachable for those new to Python – with a three-step workflow of `login()`, `search_data()`, and `download()` – while offering sufficient depth for advanced users who need direct S3 access, streaming file handles, or virtual dataset construction for large-scale analysis.

State of the field

The scholarly contribution of `earthaccess` is the *integration* of search, authentication, and access into a coherent abstraction that masks the heterogeneity of NASA Earthdata's infrastructure. No existing tool provides this end-to-end workflow across the full breadth of NASA Earthdata. Rather than reinventing query or filesystem libraries, `earthaccess` composes and extends existing open-source tools (`python-cmr` (NASA, 2026) for search, `fsspec` (fsspec Contributors, 2026) and `s3fs` (s3fs Contributors, 2026) for file I/O, and `VirtualiZarr` (Nicholas et al., 2026) and `kerchunk` (kerchunk Contributors, 2026) for virtual datasets), and contributes the NASA-specific domain knowledge (DAAC configurations, credential endpoints, authentication flows, and the mapping between HTTPS and direct-S3 access links) that binds them into a usable data access layer.

Several tools exist for accessing NASA Earth science data, each addressing only part of the workflow, or addressing the full workflow for only a subset of NASA's data holdings:

- **python-cmr** (NASA, 2026) handles the search step (step 1 of the workflow in the Statement of need) by providing a Python wrapper around the CMR API for dataset and granule queries. `earthaccess` builds on `python-cmr`, extending it with a built-in registry of NASA's DAACs (mapping each DAAC to its CMR provider IDs and S3 credential endpoints), cloud-hosting filters, and rich result objects that simplify working with metadata (step 2). However, `python-cmr` does not handle authentication,

DRAFT

data download, or cloud access (steps 3–5) – key areas where we observed workshop participants facing workflow difficulties.

- **asf_search** (Alaska Satellite Facility Discovery Team, 2026) sits between `python-cmr` and `earthaccess` in scope: it focuses on search and discovery but also handles authentication and access (workflow steps 1–5). It emerged around the same time as `earthaccess`, from similar access challenges. Unlike `earthaccess`, which is mission- and domain-agnostic, `asf_search` is tailored to synthetic aperture radar (SAR) data, adding domain-specific tooling and functionality.
- **icepyx** (Scheick et al., 2023) provides a mission-specific solution to the general problem solved by `earthaccess`: search, discovery, authentication, and access to ICESat-2 (Markus et al., 2017) data products. Established about a year before `earthaccess`, it handles workflow steps 1, 2, and 5 on its own and now leverages `earthaccess` to authenticate when users perform an action that requires logging in.
- **earthdatalogin** (Boettiger, 2025) provides similar authentication and access functionality (steps 3–5) for the R programming ecosystem. The two projects share a common motivation and serve as complementary tools for their respective language communities.

Software design

`earthaccess` is organized into four core components, each encapsulating a distinct stage of the data access workflow:

- **Authentication**: Manages the lifecycle of NASA Earthdata Login credentials, supporting environment variables, `.netrc` files, and interactive prompts. Once authenticated, the library creates HTTP sessions that safely handle NASA's cross-domain redirects, and it retrieves and renews temporary AWS S3 credentials for direct S3 access.
- **Search**: Adds DAAC-aware search and access (resolving DAAC names to CMR provider IDs), cloud-hosting detection, and deep-paging support. Query results (powered by `python-cmr`) are wrapped in rich objects that preserve the full metadata response while exposing convenient accessors for data links, spatial footprints, and formatted citations.
- **Access**: Presents a single interface over both access mechanisms – HTTPS and direct S3 reads – so that the same user code works from a local workstation or from within AWS, with the mechanism selectable by the user. Files can be opened as `fsspec`-compatible file-like objects for streaming into libraries such as `xarray` (Hoyer & Hamman, 2017), or downloaded to disk with parallel, fault-tolerant transfers. Because `earthaccess` returns ordinary Python file-like objects and makes no assumptions about the underlying file format, it interoperates directly with the wider scientific Python ecosystem – including the PyData and Pangeo tool stacks – rather than requiring format- or mission-specific readers.
- **Virtual datasets**: Leverages NASA's DMR++ sidecar metadata files (OPeNDAP, 2026) to construct virtual Zarr stores via `VirtualiZarr` (Nicholas et al., 2026) or `kerchunk` (kerchunk Contributors, 2026), enabling lazy, chunk-level access to archival HDF5/NetCDF4 data without downloading or reformatting files. For example, a researcher can extract a single variable across thousands of files by reading only the relevant byte ranges from NASA's cloud storage, with minimal local resource usage. These features are available as optional dependencies to keep the core library lightweight.

Several deliberate design decisions shape the library:

Build on existing libraries. `earthaccess` composes the existing open-source tools described in the State of the field rather than reimplementing their functionality. The library's unique contribution is the NASA-specific integration layer that binds these tools together. For example, retrieving NASA's TEMPO Level-3 ozone product (Liu, 2026) directly from the Earthdata

DRAFT

Cloud with `boto3` (Amazon Web Services, 2026), `requests`, and `s3fs` requires about two dozen lines of code (excluding imports) to set up Earthdata Login authentication, request and periodically refresh temporary S3 credentials, and serially transfer each granule. The equivalent `earthaccess` workflow is about four lines (`login()`, `search_data()`, `open()`, and passing the result to `xarray`) and additionally handles CMR-based discovery. Because that integration also leverages `fsspec` for byte-range access, streaming a single variable across 28 granules and computing a summary statistic completed in roughly 45 seconds end-to-end, compared with roughly 78 seconds to download all 6.7 GB of those granules serially beforehand (medians of three runs on the NASA-Openscapes `us-west-2` JupyterHub). Constructing or reading virtual data stores over the same files with `VirtualiZarr` (Nicholas et al., 2026), which uses precomputed chunk locations (e.g., from DMR++, kerchunk, or icechunk metadata) instead of reading each file's internal HDF5 metadata, can reduce access time further. The same pattern applies to other products with large numbers of granules, such as ICESat-2 ATL06 land-ice height data (Smith et al., 2025), and is most pronounced where an analysis touches only a small portion of each file.

Contribute upstream. When community discussions surface features that belong in a dependency, the project contributes that work upstream rather than absorbing it. Advanced CMR query capabilities were developed in `python-cmr` rather than duplicated in `earthaccess`, and aspects of DMR++ parsing and multi-file virtual dataset functionality were migrated to `VirtualiZarr` where they could benefit the wider community. This upstream-first discipline prevents scope creep, reduces maintenance burden, and strengthens the broader ecosystem that `earthaccess` depends on.

Infrastructure-agnostic access. The same user code works whether the computation runs in the cloud or on a local workstation, without requiring code changes; users who want direct S3 access can request that optimization explicitly. This reflects the reality that researchers are at varying stages of cloud adoption.

Flat and functional top-level API. All primary operations are exposed as module-level functions (e.g., `earthaccess.login()`, `earthaccess.search_data()`, `earthaccess.download()`), minimizing conceptual overhead for new users while keeping the underlying object-oriented classes accessible for advanced use cases.

Research impact statement

`earthaccess` has established itself as foundational infrastructure for NASA Earth science data access. Concrete evidence of its impact includes:

Peer-reviewed publications. `earthaccess` has been used in published research, including studies on multi-sensor drought observations in forested environments (Andreadis et al., 2024) and tidal bore detection using SWOT satellite data (Arildsen et al., 2025). Several other research studies and geospatial technology publications utilize libraries dependent on `earthaccess` (e.g., Scheick et al., 2025; Widlansky & Komar, 2025; Xiahou et al., 2026).

Community adoption. The library is a dependency of over 270 public GitHub repositories (as of 28 October 2026), spanning data analysis workflows, Jupyter Notebook tutorials, and downstream libraries. It is distributed through both PyPI and conda-forge and installed and used in cloud-hosted Jupyter environments provided by NASA and partner organizations. As one example of downstream adoption, `icepack` – a finite element library for ice sheet and glacier modeling (Shapero et al., 2021) – replaced its hand-written NSIDC DAAC data-fetching routines with `earthaccess` calls, eliminating hard-coded URLs and custom authentication logic. Across NASA data access tutorials, `earthaccess` typically replaces tens of lines of setup code in each tutorial with a few function calls, increasing user accessibility and reducing the overhead to "get started" working with NASA data.

Multi-institutional development. Contributors span NASA's Distributed Active Archive Centers

DRAFT

(DAACs) (including ASDC, ASF, GES DISC, LP DAAC, NSIDC DAAC, OB.DAAC, ORNL DAAC, and PO.DAAC) as well as other federal and academic institutions (USGS, NOAA, University of New Hampshire, University of Maryland, UC Berkeley), private industry (Coiled, Development Seed), and independent open-source contributors. This breadth reflects both the library's relevance across domains and the health of its contributor community.

Integration with the NASA ecosystem. `earthaccess` is featured in the official NASA Earthdata tools catalog (https://www.earthdata.nasa.gov/data/tools/earthaccess) and in NASA Earthdata tutorials. It has been presented at multiple large professional meetings – including several American Geophysical Union (AGU) Annual Meetings and Earth System Information Partnership (ESIP) meetings – and was the subject of a NASA ESDS Tech Spotlight presentation (see https://earthaccess.readthedocs.io homepage for links to recent slides and recordings). The documentation includes executable code examples demonstrating workflows with ICESat-2, EMIT, TEMPO, SMAP, and other missions, providing reproducible entry points for researchers.

AI usage disclosure

The `earthaccess` project has adopted a contributor AI policy (https://earthaccess.readthedocs.io/en/latest/contributor/ai-policy/) that requires disclosure of AI assistance, holds contributors responsible for understanding and being able to explain any code they submit, and prohibits fully autonomous AI-generated pull requests. The version of `earthaccess` described here was developed with minimal generative AI assistance: i.e., only a few contributions, made before the policy was adopted, used GitHub Copilot for code-review suggestions, changelog text, and CI configuration. Subsequent development may use AI tools under the terms of that policy.

Preparation of this manuscript was assisted by Claude (Sonnet 4.6, Opus 4.6, Opus 5, and Opus 5.5; Anthropic) using the repository source code, documentation, commit history, and project metadata as context. Assistance was used for manuscript drafting and editorial revision. All content was reviewed, revised, and verified for accuracy by the authors, who bear full responsibility for the submitted work.

Acknowledgements

The development of `earthaccess` was supported by NASA's Earth Science Data Systems (ESDS) program through the NASA Openscapes project (NASA award #20-TWSC20-2-0003 "Openscapes: Enabling a new era of science on the cloud, in response to the 2020 NASA ROSES Element E2", Julie Lowndes and Erin Robinson). We thank NASA Openscapes for the community workshops, collaborative working environment, and people-first approach that have motivated and continue to support this work. We are grateful to the National Snow and Ice Data Center (NSIDC) for hosting the repository during its initial development and to all contributors who have shaped `earthaccess` through contributing code, documentation, issue reports, and community engagement. People contribute to `earthaccess` as adopters and developers as part of their jobs across the NASA ecosystem as well as outside of it, which is critical to making this shared resource successful. We appreciate everyone who advocates for and uses approved time to make this possible. We also thank Allison Horst for the `earthaccess` artwork and Daniel Kaufman for leading the preparation of this submission.

References

Alaska Satellite Facility Discovery Team. (2026). *asf_search: Python wrapper for the Alaska Satellite Facility (ASF) SearchAPI*. https://github.com/asfadmin/Discovery-asf_search

DRAFT

Amazon Web Services. (2026). *Boto3: The AWS SDK for Python*. https://github.com/boto/boto3

Andreadis, K. M., Meason, D., Corbett-Lad, P., Höck, B., & Das, N. (2024). Hydrologic consistency of multi-sensor drought observations in forested environments. *Remote Sensing*, *16*(5), 852. https://doi.org/10.3390/rs16050852

Arildsen, R. L., Andersen, O. B., Nilsson, B., & Ludwigsen, C. B. (2025). Tidal bore revealed by SWOT: A case study from the Severn river. *Scientific Reports*, *15*, 22776. https://doi.org/10.1038/s41598-025-99597-2

Boettiger, C. (2025). *earthdatalogin: NASA Earthdata Login utilities for R*. https://doi.org/10.32614/CRAN.package.earthdatalogin

fsspec Contributors. (2026). *filesystem_spec: Filesystem interfaces for Python*. https://github.com/fsspec/filesystem_spec

Hoyer, S., & Hamman, J. (2017). xarray: N-D labeled arrays and datasets in Python. *Journal of Open Research Software*, *5*(1), 10. https://doi.org/10.5334/jors.148

kerchunk Contributors. (2026). *kerchunk: Cloud-friendly access to archival data*. https://github.com/fsspec/kerchunk

Liu, X. (2026). *TEMPO gridded ozone total column V04 (PROVISIONAL)*. NASA Langley Atmospheric Science Data Center Distributed Active Archive Center. https://doi.org/10.5067/IS-40e/TEMPO/O3TOT_L3.004

Lowndes, J. S. S., Froehlich, H. E., Horst, A., Jayasundara, N. C. Y., Pinsky, M. L., Stier, A., Therkildsen, N. O., & Wood, C. L. (2019). Supercharge your research: A ten-week plan for open data science. *Nature*. https://doi.org/10.1038/d41586-019-03335-4

Lowndes, J. S. S., & Robinson, E. (2026). *NASA Openscapes*. https://nasa-openscapes.github.io

Markus, T., Neumann, T., Martino, A., Abdalati, W., Brunt, K., Csatho, B., Farrell, S., Fricker, H., Gardner, A., Harding, D., Jasinski, M., Kwok, R., Magruder, L., Lubin, D., Luthcke, S., Morison, J., Nelson, R., Neuenschwander, A., Palm, S., … Zwally, J. (2017). The Ice, Cloud, and land Elevation Satellite-2 (ICESat-2): Science requirements, concept, and implementation. *Remote Sensing of Environment*, *190*, 260–273. https://doi.org/10.1016/j.rse.2016.12.029

NASA. (2026). *Python CMR: Python wrapper to the NASA Common Metadata Repository (CMR) API*. https://github.com/nasa/python_cmr

National Aeronautics and Space Administration. (2026a). *Common Metadata Repository (CMR)*. https://cmr.earthdata.nasa.gov/search/site/docs/search/api.html

National Aeronautics and Space Administration. (2026b). *Earthdata Login*. https://urs.earthdata.nasa.gov

National Aeronautics and Space Administration. (2026c). *ESDS Data Metrics*. https://www.earthdata.nasa.gov/about/data-metrics

National Aeronautics and Space Administration. (2026d). *NASA Earthdata*. https://www.earthdata.nasa.gov

Nicholas, T., Hagen, N., Harkins, S., Barciauskas, A., Jones, M., Signell, J., Nag, A., Hidalgo, G., Augspurger, T., & Abernathey, R. (2026). *VirtualiZarr*. https://github.com/zarr-developers/VirtualiZarr

OPeNDAP. (2026). *OPeNDAP building and testing DMR++ documents*. https://opendap.github.io/DMRpp-wiki/DMRpp.html

s3fs Contributors. (2026). *s3fs: Pythonic file interface to S3*. https://github.com/fsspec/s3fs

DRAFT

Scheick, J., Leong, W. J., Bisson, K., Arendt, A., Bhushan, S., Fair, Z., Hagen, N. R., Henderson, S., Knuth, F., Li, T., Liu, Z., Piunno, R., Ravinder, N., Setiawan, L. "Don"., Sutterley, T., Swinski, J. P., & Anubhav. (2023). icepyx: Querying, obtaining, analyzing, and manipulating ICESat-2 datasets. *Journal of Open Source Software*, *8*(84), 4912. https://doi.org/10.21105/joss.04912

Scheick, J., Piunno, R., Fair, Z., Tilling, R., Di Bella, A., Abib, N., & Bisson, K. M. (2025). Software to enable ocean discoveries: A case study with ICESat-2 and Argo. *Geoscience Data Journal*, *12*(3), e291. https://doi.org/10.1002/gdj3.291

Shapero, D., Badgeley, J. A., Hoffman, A. O., & Joughin, I. R. (2021). icepack: A new glacier flow modeling package in Python, version 1.0. *Geoscientific Model Development*, *14*(7), 4593–4616. https://doi.org/10.5194/gmd-14-4593-2021

Smith, B., Adusumilli, S., Csathó, B., Felikson, D., Fricker, H., Gardner, A., Holschuh, N., Lee, J., Nilsson, J., Paolo, F., Siegfried, M., Sutterley, T., & the ICESat-2 Science Team. (2025). *ATLAS/ICESat-2 L3A land ice height, Version 7*. NASA National Snow and Ice Data Center Distributed Active Archive Center. https://doi.org/10.5067/ATLAS/ATL06.007

Widlansky, M. J., & Komar, N. (2025). Building an intelligent data exploring assistant for geoscientists. *Journal of Geophysical Research: Machine Learning and Computation*, *2*(3), e2025JH000649. https://doi.org/10.1029/2025JH000649

Xiahou, Y., Brewer, M., Hulbe, C., & Stevens, C. (2026). Hydrographic variability in the central Ross Ice Shelf cavity and the implications for ocean circulation. *Journal of Geophysical Research: Oceans*, *131*(1), e2025JC023511. https://doi.org/10.1029/2025JC023511

DRAFT